

KVFabric：面向大模型推理服务的 KV Cache 生命周期管理系统

期末结题报告：最终设计与测试分析

KVFabric 是中国科学技术大学 OSH-2026 操作系统课程大作业项目。项目围绕大模型推理服务中的 KV Cache 资源管理展开，基于 vLLM 0.22.1 构建控制面 overlay，将原本主要由 free queue、ref count 和 prefix hash 隐式管理的 KV block 抽象为带生命周期、共享前缀关系、重算代价、请求语义和调度反馈的系统资源对象。

在系统实现上，KVFabric 保留 vLLM 的 PagedAttention 数据面、OpenAI 兼容服务入口和原有请求执行流程，在 BlockPool、KVCacheManager、SingleTypeKVCacheManager、Scheduler、OpenAI serving、Prometheus metrics 和长测工具链中接入 lifecycle side table、Prefix Family 元数据、evicted shadow、rebuilt-from-eviction 反馈、hint-aware admission、shared-aware eviction、family-protect、scheduler affinity、latency guard、SLO goodput 统计、JSONL 事件流、dashboard 和 replay 工具。

最终项目已经形成从本地 Qwen3.5-2B smoke test 到远程 2 x RTX 3090 上 Qwen3.5-9B 长周期实验的完整闭环。最终 12h 矩阵覆盖高压共享前缀吞吐、前后台混部的交互延迟、企业混合流量、低复用保护等场景。实验结果显示，KVFabric 在高压、稳定共享前缀和冷热混合流量中能明显减少错误驱逐后的重建，提升 prefix cache 使用质量，并在 SLO 边界场景下提高 goodput；在普通低频混合流量和低复用场景中保持低干预，并通过 admission 控制减少 cache churn 和排队压力。

本报告按照参考结题报告的正文型结构组织，先说明立项依据和核心挑战，再展开最终系统设计、代码实现、接手后的迭代过程、实验平台与请求生成、最终测试结果、调试工具和项目总结。报告中的实验数值来自仓库归档的 final 12h run root、summary、metrics.json 和 lifecycle metrics。

小组成员以及分工

成员分工

成员	主要工作
周家润	生命周期框架设计，远程服务器部署和长周期实验，实验编写，调试工具开发，dashboard/replay，实验结果分析和策略迭代。
赵天翔	vLLM 控制面阅读，lifecycle side table 封装，BlockPool/KVCacheManager/Scheduler 策略实现，admission、eviction、scheduler 和 metrics 接入。
王允	实验设计、长对话压测、日志维护、场景说明和阶段材料整理。

1. 立项依据

1.1 研究背景

大模型推理服务的成本和性能瓶颈很大一部分来自 KV Cache。对于 decoder-only Transformer，每生成一个 token 都需要依赖之前 token 的 key/value 张量。服务端为了避免重复计算，会把这些张量按 block 形式保存在显存中。请求越长、并发越高、上下文复用越多，KV Cache 对吞吐、延迟、显存容量和调度公平性的影响越明显。

vLLM 通过 PagedAttention 把 KV Cache 管理成 block，并用 prefix caching 复用相同前缀。该机制已经解决了许多传统连续 KV 分配的碎片问题，也让相同系统提示词、RAG 模板、多轮对话历史和 agent 工具协议可以直接复用前缀 prefill 结果。实际服务中，问题逐步从“能不能缓存”转向“缓存哪些内容更有价值”“容量紧张时先淘汰什么”“热前缀对应的请求能否及时调度”“如何解释一次优化到底改好了哪类请求”。

从操作系统角度看，KV Cache 与传统页缓存、工作集、置换策略和准入控制存在天然对应关系。block 是显存中的页；prefix family 类似共享映射和工作集；cold RAG、burst 请求和 decode-heavy 后台作业类似短期噪声；SLO goodput 和前台请求延迟对应系统服务质量。KVFabric 选择在 vLLM 的真实控制面中实现这些思想，使课程项目可以落到可运行代码和可复现实验上。

1.2 原生 vLLM 控制面的特点

原生 vLLM 的 KV Cache 控制面非常简洁。请求进入 scheduler 后，KVCacheManager 计算 prompt 对应的 block hash，利用 prefix cache map 查找最长已缓存前缀；命中的 block 被 touch，引用计数增加；未命中的 token 继续 prefill 并申请新 block；请求完成后 ref count 归零，block 重新进入 free queue。空间不足时，free queue 中仍带 hash 的 cached block 会按 LRU 语义从 prefix cache 中移除。

这一设计适合高性能通用推理服务，但它默认把 full block cache entry 作为相对被动的缓存页处理。控制面很少记录 block 的历史价值，驱逐后也缺少“同 hash 后续是否又被重建”的反馈。对于普通短请求，这样的简洁性有优势；对于长期共享前缀、冷热混合和多租户服务，单纯 LRU 很难区分长期有价值的共享主干和短期只出现一次的冷长尾。

1.3 核心技术挑战

第一个挑战是可观测性。只看 prefix hit rate、tok/s 和平均 latency，无法解释某一次 A/B 结果来自前缀复用、排队改善、admission 限制、scheduler promotion，还是 workload drift。项目需要把 block、request、family 和 policy 的状态记录下来，形成可复盘的证据链。

第二个挑战是共享关系识别。vLLM 原生 exact prefix caching 已经有 block hash 和 parent hash，但没有把多个请求共享的前缀组织成可解释的 family。KVFabric 需要在不重写 prefix cache 结构的情况下，记录 root、parent、depth、hit、branch、regret 等控制面元数据，为共享主干保护提供依据。

第三个挑战是策略边界。强驱逐排序可能增加分配热路径开销，过度保护共享主干可能污染 cache，过强 scheduler promotion 会牺牲 decode-heavy 和低复用请求。项目必须让策略参数化，保留 LRU 快路径和低干预路径，并用 class-level 指标审计收益和代价。

第四个挑战是实验公平性。闭环 fixed-concurrency 会让更快的策略在同样时间内完成更多请求，开环 trace 又更敏感于排队与 backlog。长测运行时间长，远程服务器可能出现 partial、stalled、网络断

开和单个 policy 失败。项目需要固定 seed、配置快照、trace hash、run state、heartbeat、summary 和 dashboard，确保结果可以复现和验收。

1.4 项目目标和创新点

- 把 KV block 从隐式缓存页提升为带生命周期、共享关系、重算代价和保留价值的控制面对象。
- 在 vLLM 0.22.1 的真实 OpenAI serving 路径上实现 request hint、admission、eviction、scheduler 和 metrics 联动。
- 用 evicted shadow 和 rebuilt-from-eviction 衡量错误驱逐，避免只用总体 hit rate 解释缓存质量。
- 构建 tenant、family、session、turn、phase 驱动的真实化 trace，并支持按 scheduled_at_seconds open-loop replay。
- 完成本地 smoke、中等远程调参、最终 12h 矩阵和 dashboard/replay 的完整实验闭环。

2. 总体设计

2.1 框架模块设计总览

KVFabric 的总体架构可以分为服务入口、调度层、KV Cache 管理层、BlockPool 层、生命周期控制面和实验观测层。请求仍通过 vLLM OpenAI-compatible `/v1/chat/completions`` 进入，模型执行仍由 vLLM engine 和 worker 完成。KVFabric 的主要工作发生在 Python 控制面：它观察并影响 block cache 写入、驱逐候选选择、waiting queue 调度和指标输出。

框架设计遵循“数据面保持稳定、控制面增加状态、实验面闭环验证”的原则。数据面继续由 vLLM engine、worker、PagedAttention 和 GPU KV tensor 负责；控制面 overlay 记录 request、block 和 family 的状态，并在 admission、eviction、schedule 三个位置施加轻量策略；实验面负责把 workload、run state、Prometheus、JSONL、summary 和 dashboard 连接起来。

图 1 KVFabric 本体代码整体框架设计图

KVFabric 本体代码整体框架设计图

vLLM 0.22.1 原生路径 + KVFabric 控制面 overlay + 远程 9B 长测闭环

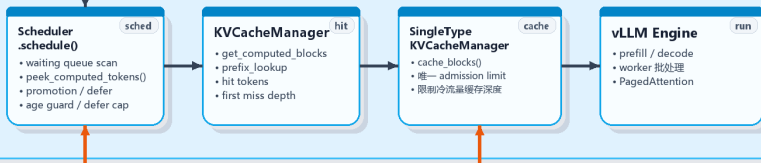
A. vLLM 原生执行栈：保留推理语义，插入轻量控制点

请求从 OpenAI serving 进入，调度、prefix lookup、cache blocks 和 BlockPool 仍沿 vLLM 主路径执行

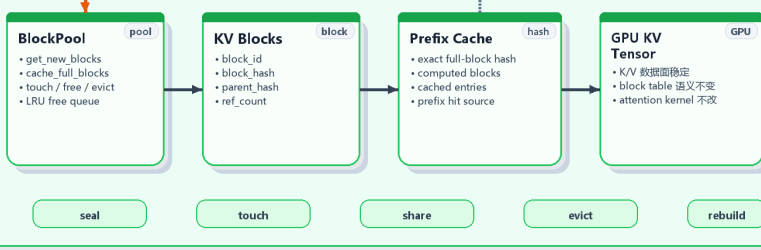
1 请求语义入口



2 调度与 prefix lookup 控制路径



3 KV Cache 数据面与生命周期事件源



底层保持稳定：GPU KV tensor、PagedAttention kernel、vLLM block table 语义不被重写

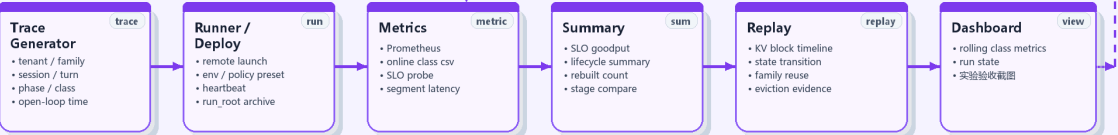
B. KVFabric Overlay 控制面

side table + family index + controller



C. 远程长测、指标汇总与可视化闭环

2 x RTX 3090 / Qwen3.5-9B / trace replay / run state / dashboard



实线：请求执行与策略反馈



虚线：hint / lifecycle / family / metrics 采集



H/E/F/S/C/B：对应接入点编号

图 1 展示了 KVFabric 与 vLLM 的模块边界。上层客户端或 load generator 只看到 OpenAI 兼容接口和普通 HTTP headers; vLLM 的请求调度、prefix lookup、block 分配、prefill/decode 仍按原路径执行; KVFabric 在控制面维护 side table、family index 和 controller, 并把策略结果反馈到 cache_blocks、get_new_blocks 和 scheduler waiting queue。图中虚线表示事件采集路径, 实线表示请求执行和策略反馈路径。

该架构的边界比较清晰: 模型权重、attention kernel、PagedAttention block 数据布局和分布式执行不被改写; 新增逻辑集中在 overlay 文件中, 默认可以关闭。这样既保留 vLLM 的性能基础, 也方便与原生 LRU 行为做 A/B 对照。

vLLM 与 KVFabric 的职责对照

层次	vLLM 原生职责	KVFabric 增加的职责
数据面	PagedAttention block、KV tensor、worker 执行	不改写 tensor 数据面, 只读取和记录生命周期事件
缓存控制面	prefix hash lookup、free queue、LRU cached block 驱逐	side table、family metadata、retain score、admission、eviction deferral
调度层	FCFS/priority、token budget、running/waiting 管理	hit-aware promotion、latency-protected promotion、age guard、defer cap
服务入口	OpenAI-compatible request、trace headers	request class、tenant、family、session、turn、SLO 等 hint plumbing
观测层	基础 throughput、latency、prefix cache metrics	lifecycle JSONL、rebuilt、class metrics、SLO goodput、dashboard/replay

2.2 运行时模块边界

KVFabric 的运行时模块可以按“请求语义、缓存对象、策略控制、指标输出”四类理解。请求语义来自 serving header 和 trace metadata; 缓存对象来自 vLLM 的 block hash、parent hash、ref count 和 free queue; 策略控制由 controller 参数决定; 指标输出面向长测验收和调试。模块之间通过显式事件和 side table 连接, 避免把策略状态散落在多个 vLLM 类中。

运行时模块分层

模块层	核心对象	输入	输出
Serving hint 层	KVFabricRequestHints	x-kvfabric-* headers、trace request id、SLO	request_class、tenant、family、session、turn、priority、expected reuse
Prefix lookup 层	RequestMeta	block hash、prompt tokens、computed blocks	prefix_hit_tokens、first_miss_depth、eligible hit 统计
Block lifecycle 层	LifecycleBlockMeta	alloc/seal/touch/free/evict 事件	state、hit/share、recompute cost、retain score
Prefix family 层	PrefixNodeMeta / PrefixFamilyMeta	root_hash、parent_hash、children、depth	family hit、branch、regret、protected_depth
Controller 层	KVFabricControlConfig	Admission/Eviction/Schedule/SLO strengths	admission limit、victim ranking、promotion score
Metrics 层	JSONL / Prometheus / summary	事件流、online metrics、rolling metrics	SLO goodput、class metrics、dashboard、replay

2.3 运行时模块协作

运行时协作的关键在于事件方向和决策方向分开。BlockPool、KVCacheManager 和 Scheduler 把事件送入 tracker；tracker 维护 side table 和 family index；controller 根据当前压力、请求 hint 和历史复用证据给出 admission、eviction 和 scheduling 建议；vLLM 原模块再按建议执行具体动作。

图 2 运行时控制面模块协作

事件来源	OpenAI serving request hints	Scheduler queue/promote/defer	KVCacheManager prefix lookup	BlockPool block events
状态中心	KVFabricLifecycleTracker LifecycleBlockMeta / RequestMeta / EvictedShadow PrefixFamilyIndex / Controller counters			
决策反馈	Admission limit cache depth	Eviction retain score family protect	Schedule hit-aware score latency guard	Metrics JSONL / Prometheus class summary
事件流: serving/scheduler/KV manager/block pool → tracker		决策流: tracker/controller → admission/eviction/schedule/metrics		

这个协作方式有两个好处。第一，BlockPool 和 Scheduler 不需要各自维护一套重复状态，降低双写风险。第二，LRU baseline 可以开启观测但把 controller strength 设为 0，从而在同一套日志和指标体系下比较原生行为与 KVFabric 行为。

2.4 Lifecycle side table

Lifecycle side table 是 KVFabric 的中心数据结构。它以 block id 为索引记录当前 block 的 hash、prefix depth、ref count、hit count、share degree、branch factor、recompute cost tokens、state、family id、root hash、parent hash、protected depth 和最近事件时间。请求侧另有 RequestMeta，记录 request id、prompt tokens、prefix hit tokens、request class、hint、defer/admission 计数。被驱逐 block 会进入 EvictedShadow，保留 hash、深度、命中、共享、retain score 和驱逐时间。

Lifecycle side table 关键字段

字段组	代表字段	作用
身份与位置	block_id, block_hash, prefix_depth	定位 block 在 prefix 链中的位置，估计重算代价
复用证据	hit_count, share_degree, ref_count	判断 block 是否被多请求复用或近期命中
结构证据	family_id, root_hash, parent_hash, branch_factor	把 block 放回 prefix family 结构中理解
策略证据	recompute_cost_tokens, retain_score, protected_depth	为驱逐和保护提供可解释的评分
状态机	FREE, ACTIVE, SEALED, SHARED, COOLING_WARM, COOLING_HOT, EVICTED	解释 block 从分配到驱逐的生命周期
反馈	EvictedShadow, rebuilt_from_eviction	识别被驱逐 hash 后续重建，衡量错误驱逐

这个 side table 没有实现 token 级 trie，也没有记录每个 child branch 的完整权重分布。当前系统使用 vLLM 已有的 full-block hash 链作为基础，记录 block 级近似共享信号。这样的选择符合课程项目的工程边界：在真实 vLLM 控制面内保持低侵入，同时把最需要解释的生命周期和重建反馈记录下来。

2.5 Prefix Family 元数据

Prefix Family 用于描述一组具有共同 root 的 block-aligned prefix。vLLM 的 block hash 计算中已经包含 parent hash，KVFabric 利用 sealed full block 时可见的当前 hash、parent hash 和 root hash，维护 PrefixNodeMeta 和 PrefixFamilyMeta。一个 family 中，浅层 block 往往是系统 prompt、企业流程模板、RAG 固定头、多轮会话历史或 agent 协议；深层 block 更可能是请求私有尾部。

Family 元数据记录 hit_count、sealed_blocks、evicted_blocks、rebuilt_blocks、regret_count、max_depth 和 protected_depth。family value 越高，浅层共享主干越适合保护。branch factor 用于近似表示分叉程度：如果同一 parent 下出现多个 child hash，说明该 prefix 后面存在多个分支，浅层共享价值更高。

2.6 Hint-aware serving

KVFabric 在 OpenAI serving 入口增加请求 hint plumbing。load generator 通过 HTTP header 发送 `x-kvfabric-request-class`、tenant、family、session、turn-index、cache-priority、expected-reuse、phase、burst、SLO ms 和 hint confidence。serving.py 与 tracing/utlis.py 将这些 header 保留下来，scheduler 再转成 KVFabricRequestHints 写入 tracker。

请求 hint 与策略用途

Hint	示例	策略用途
request class	durable_hot_family, cold_rag_unique, decode_heavy_background	区分前台、后台、冷请求和长输出请求
tenant/family/session	tenant_3, family_42, session_18	建立业务级复用关系和多轮会话关系
expected reuse	durable, transient, none, unknown	决定 admission fraction 和 scheduler score
cache priority	high, normal, low, bypass	控制低价值请求是否写入 prefix cache
turn index	0, 1, 2, ...	识别多轮后续 turn 的共享历史价值
SLO ms	60000, 90000, 120000	计算 SLO goodput 和 latency promotion 条件

这些 hint 属于 KVFabric 实验控制面，OpenAI 接口和 vLLM 原生接口本身没有定义这组字段。它们通过普通 HTTP header 传递，不改变 ChatCompletions payload 的基础格式。真实产品中，这类信息可以由网关、业务服务或 agent runtime 提供；在本项目中，trace generator 和 load generator 负责按可复现规则生成。

2.7 统一 Controller

项目后期把策略表达收敛为统一 controller，减少对 `shared\_aware`、`kvfabric\_latency` 这类离散名称的依赖。controller 包含 Admission、Eviction、Schedule、SLO protect、Hint trust 和不同请求类别的 cache fraction。stage-local preset 只表示当前实验重点，不代表不同代码版本。

Controller 参数含义

参数	含义	起效位置
Admission strength	控制低复用、bypass、cold、unknown 请求写入 prefix cache 的比例	SingleTypeKVCacheManager.cache_blocks()
Eviction strength	控制 retain score、shared-aware 或 family-protect 对 LRU victim 的影响	BlockPool.get_new_blocks()
Scheduler strength	控制 waiting queue scan、positive promotion 和 hit-aware score 强度	Scheduler.schedule()
SLO protect strength	控制接近 SLO 或 latency-protected class 的 promotion 条件	Scheduler latency promotion

参数	含义	起效位置
Hint trust	控制 request hint 对 admission/scheduler 分数的权重	KVFabricLifecycleTracker scoring
Cache fractions	durable、transient、low-reuse、bypass、cold 等类别允许缓存的比例	admission limit

2.8 Metrics、JSONL 与 replay

KVFabric 同时保留 Prometheus 指标和 JSONL 事件流。Prometheus 适合看在线曲线和长期聚合，JSONL 适合复盘某个 block、request 或 family 的生命周期。summary 脚本把两者合并成 stage 级表格，dashboard 负责实时查看 run state、heartbeat、rolling metrics、class metrics 和 lifecycle replay。

图 3 实验观测与长测闭环



JSONL 事件包括 tracker_initialized、request_hints_observed、prefix_lookup、block_allocated、block_sealed、block_touched、ref_count_changed、cache_admission_limited、request_promoted、request_latency_promoted、request_deferred、request_finished、eviction_candidates_ranked、block_evicted 和 lifecycle_reset。它们可以回答三类问题：请求语义是否进入 vLLM、prefix cache 是否命中、block 从 active 到 evicted 的路径是否符合预期。

3. 核心实现

3.1 代码结构

核心实现集中在 `vllm\_workspace/overlay`。overlay 文件覆盖 vLLM 0.22.1 的少量 Python 控制面模块，便于与 upstream 和 baseline 对照。实验代码集中在 `experiments/long\_pressure\_benchmark`，包含 trace 生成、online duration loadgen、online trace loadgen、远程部署、结果同步、summary、dashboard 和 replay。

核心文件与职责

文件	职责
vllm/v1/core/kvfabric_lifecycle.py	生命周期 side table、controller、retain score、admission、scheduler scoring、JSONL 事件
vllm/v1/core/kvfabric_family.py	Prefix Family 的 root、parent、children、hit、regret、protected depth 统计
vllm/v1/core/kvfabric_hints.py	解析 KVFabric request headers，统一 hint 字段和默认值
vllm/v1/core/block_pool.py	sealed、touch、free、evict 接入，shared-aware 和 family-protect victim 选择
vllm/v1/core/kv_cache_manager.py	prefix lookup 事件、prompt tokens、hit tokens 记录
vllm/v1/core/single_type_kv_cache_manager.py	admission limit 唯一路径，限制 full block 写入 prefix cache
vllm/v1/core/sched/scheduler.py	waiting queue scan、promotion、latency guard、defer cap
vllm/entrypoints/openai/engine/serving.py	OpenAI serving header 提取和 request trace headers 传递
vllm/v1/metrics/*	Prometheus 指标和 logger 输出扩展
experiments/long_pressure_benchmark/*	9B/27B 长测配置、loadgen、runner、summary、dashboard、replay

3.2 LifecycleTracker

KVFabricLifecycleTracker 是 overlay 的状态中心。BlockPool 初始化时创建 tracker，随后在 block allocated、sealed、touched、free、evicted、request hints、prefix lookup、request finish 等事件点调用 tracker。tracker 内部维护 block_meta、request_meta、evicted_shadow、family index、event counters 和 JSONL writer。

在 block sealed 时，tracker 会记录 block hash、parent hash、root hash 和 prefix depth，并把该 block 纳入 family index。如果同 hash 曾经出现在 evicted shadow 中，当前 sealed 事件会被标记为 rebuilt_from_eviction。这个反馈很关键，因为它把驱逐策略的错误后果从抽象 miss 变成可计数的重建事件。

在 prefix lookup 时，tracker 记录 request 的 prompt tokens、hit tokens、first miss depth 和 hint 信息。后续 scheduler scoring 会使用这些 request meta，summary 也能按 class 统计 prefix 命中和 SLO goodput。request finish 时，tracker 回收 request hints，减少长测中的控制面内存增长。

3.3 BlockPool 接入

BlockPool 是 KV block 生命周期最集中的位置。KVFabric 在 cache_full_blocks() 中记录 full block sealed，在 touch() 中记录 prefix hit 和共享引用，在 free_blocks() 中更新 ref count 和 cooling 状态，在 _maybe_evict_cached_block() 中记录 block_evicted 和 EvictedShadow。get_new_blocks() 是驱逐策略的入口。

为了控制开销，`get_new_blocks()` 保留无压力快路径：如果队头将被分配的 free blocks 没有 `block_hash`，说明没有 cached block 会被驱逐，直接按 vLLM 原逻辑取走。只有当队头包含 cached block，且 controller 表明需要干预时，才进入 shared-aware 或 family-protect 选择器。9B 实验后期更偏向 linear selector，减少候选排序和 free queue remove 开销。

3.4 Admission 路径

admission 控制最终收敛在 `SingleTypeKVCacheManager.cache_blocks()`。早期尝试在多个位置限制缓存，出现过 double-limit 风险：上层认为某些 block 已经被缓存，底层又限制写入，可能破坏 block hash 与 cached entry 的一致性。后期把实际限制统一放在 `cache_blocks()`，先调用 `cache_pressure_snapshot()`，再由 `tracker.limit_cache_blocks()` 返回允许缓存的 full block 数量。

`limit_cache_blocks()` 不会影响请求执行和模型输出，它只控制新算出的 full block 是否写入 prefix cache。durable、session、high priority 请求保留完整缓存机会；cold RAG、bypass、low-reuse 和 unknown 请求在压力下只保留 discovery/anchor block 或少量前缀。summary 中的 admission limited、saved blocks、limited hint classes 就来自这条路径。

3.5 驱逐策略

shared-aware eviction 对候选窗口计算 retain score。score 综合 hit_count、share_degree、branch_factor、prefix_depth、recompute_cost_tokens、family hit、recent regret 和 protected depth。分数高的 block 被认为更值得保留，低分 block 更适合作为 victim。family-protect 则更轻量，它沿 LRU 顺序扫描，遇到 protected block 时延后选择，普通 block 足够时先驱逐普通 block，候选不足时回退。

实验中我们没有把强驱逐排序作为所有场景的默认主路径。远程 9B 结果显示，admission 在高压吞吐和低复用 guard 中更稳定，eviction 更适合补充共享主干保护。这个结论来自多轮调参：排序窗口过大时，allocation 热路径的移动和计算开销会抵消部分收益；保护过强时，短期噪声减少了，但过期 family 也可能长期占据 cache。

3.6 Scheduler affinity 与 latency guard

只保护 block 还不够。高复用前缀所在请求如果长期排在 waiting queue 后面，cache 质量无法转化为用户可见收益。KVFabric 在 scheduler 中增加 waiting queue scan，通过 `KVCacheManager.peek_computed_tokens()` 估计候选请求 prefix hit tokens，再结合 hint、age、class、SLO 和输出长度打分。

scheduler 侧包含两类机制：positive promotion 和 latency-protected promotion。positive promotion 面向 durable/session/high reuse 请求，强调 cache affinity；latency-protected promotion 面向前台交互类请求，强调 e2e latency。两者都受 scan window、max per step、head age guard、SLO head guard、defer cap 限制。低复用、decode-heavy、background 和 bypass 请求会获得 penalty，避免在 foreground-priority 场景中被无差别提升。

多轮 latency 调试的一个重要结果是：单纯把所有接近 SLO 的请求提升会伤害整体队列。后期把 promotion 改成评分选择，并把前台类、后台类、长输出类分开统计。最终 12h interactive_latency stage 因此被定义为 foreground-priority profile，报告只对前台类做 30% 以上尾延迟改善声明，同时明确 background/decode 的代价。

3.7 实验工具链实现

长测工具链由 deploy、runner、loadgen、sync、summary、dashboard 和 replay 组成。runner 在每个 stage 内启动 LRU 与 KVFabric policy，保存 config snapshot、env snapshot、policy state、run state、heartbeat、Prometheus summary、online metrics、class metrics、segment metrics、lifecycle metrics 和 sampled raw outputs。sync 脚本只拉取必要文件，避免巨大的 raw JSONL 阻塞本地归档。

online_duration_loadgen.py 使用固定请求池和分段并发，适合构造 warmup、cold churn 和 durable revisit。online_trace_loadgen.py 按 trace 中的 scheduled_at_seconds open-loop 发送请求，用 semaphore 控制 max_in_flight，记录 send delay、service latency 和 e2e latency。两者都通过 httpx.AsyncClient 向 vLLM OpenAI 兼容服务发送 POST 请求。

4. 接手后的主要迭代过程

6 月中旬后，项目从本地短测和基础策略验证进入远程 9B 长周期实验阶段。后期迭代的主线是通过实验暴露系统问题，再回到代码和架构修改。下面按问题出现顺序整理九个较大的迭代点。

6 月中旬后的九个主要迭代问题

编号	问题	表现	改进方向
1	远程长测闭环	本地短测能验证机制，远程 9B 长周期 A/B 缺少可复现归档	建立 deploy / runner / sync / summary / run root 闭环
2	真实化 trace 与 open-loop loadgen	人工 hot/cold workload 与真实服务差距较大，A/B 可能 drift	用 tenant / family / session / turn / phase 建模并按 scheduled_at_seconds replay
3	指标体系升级	raw tok/s 和平均 latency 解释不了 class 差异和 lifecycle 收益	增加 e2e、class、segment、SLO probe 和 lifecycle summary
4	Hit-aware positive scheduler	热 prefix 对应请求仍可能排在 waiting queue 后面	引入 waiting queue scan、peek_computed_tokens() 和 request promotion
5	Fairness / latency guard	复用优先会伤害 decode-heavy、低复用和后台长输出请求	加入 age guard、defer cap、latency-protected classes
6	Run state / dashboard / replay	partial 或 stalled 容易被误判为完成，JSONL 难人工阅读	增加 run state、heartbeat、rolling metrics 和 replay/dashboard
7	Unified controller	离散策略名无法说明具体机制组合	抽象为 Admission / Eviction / Schedule / SLO protect 控制向量
8	Admission 路径重构	cold RAG 进入 cache 后再驱逐制造 churn，double-limit 会破坏一致性	在 SingleTypeKVCacheManager.cache_blocks() 做唯一 admission limit
9	Latency header plumbing 与 scoring	SLO/session/turn 丢失，promotion 过粗，latency 初期无效	修 header whitelist，构造 queue pressure，改成 class/hint 惩罚评分

4.1 远程长测闭环

六月 8 日后，项目准备在远程 2 x RTX 3090 服务器上开展长周期实验。最初的本地 smoke test 只能证明代码路径可运行，无法支撑 9B 模型、长时间压力和 A/B 结论。远程实验如果只保存零散日志，后期很难回答某次结果对应的代码版本、环境变量、配置、trace 和 policy。

改进是建立 run root 闭环。每个实验 stage 都生成独立目录，里面保存 config snapshot、env snapshot、trace hash、policy state、heartbeat、server log、loadgen metrics、Prometheus summary、lifecycle metrics 和 generated summary。远程脚本负责部署 overlay、启动 vLLM、运行 loadgen、同步结果和归档摘要。这个闭环让后续所有大实验都能回到具体文件，摆脱口头记录带来的不确定性。

4.2 真实化 trace 与 open-loop loadgen

早期 workload 主要是人工 hot/cold 组合，能触发机制，但像实验室造出的压力。真实服务中，请求来自不同租户、客户、会话和业务阶段；同一 session 会多轮回访，同一 family 会形成共享模板，burst 和 background job 会穿插出现。为了让实验更接近服务场景，我们设计了 tenant、family、session、turn、phase、request_class 的 trace schema。

open-loop replay 是这一轮的关键改进。trace 中每条请求带 scheduled_at_seconds，loadgen 按绝对时间发送请求。LRU 和 KVFabric 使用同一份 trace，服务端快慢不会改变请求到达序列。这样可以把实验分成两类：duration workload 用于吞吐和容量压力，open-loop trace 用于排队、SLO、前后台混部和低复用 guard。

4.3 指标体系升级

远程初期只看 raw tok/s、平均 latency 和 prefix hit，很难解释策略效果。比如某个策略 tok/s 提升，可能是完成了更多短请求，也可能是 SLO 内 token 更多；某个 p95 下降，可能来自前台类改善，也可能来自后台请求减少。为此 summary 增加了 service latency、e2e latency、class metrics、segment metrics、SLO probe、SLO goodput、admission saved blocks、rebuilt-from-eviction、warm-family hit 和 scheduler promotion 计数。

service latency 表示请求发到 vLLM 后的服务处理时间；e2e latency 进一步包含 open-loop 发送延迟和排队影响，更适合解释 scheduler 与拥塞场景。SLO goodput 的口径是 SLO 内成功完成的 token 数除以计分秒数。prefill throughput stage 中保留多个 SLO probe，并在报告中固定展示 40s selected probe，raw JSON 中仍保留所有阈值。

4.4 Hit-aware positive scheduler

实验发现，即使 admission 和 eviction 已经保护了部分热 prefix，热 prefix 对应的请求如果排在 waiting queue 后面，仍然无法及时利用缓存。于是 scheduler 加入 waiting queue scan 和 peek_computed_tokens()。调度时扫描前若干个 waiting request，估计它们当前能命中的 prefix tokens，再结合 request hints 计算 promotion score。

这一改动把 cache 质量转成请求级收益。之前系统只能说“某些 block 应该保留”，现在可以进一步把“能利用这些 block 的请求”提前安排。promotion 有 max per step 和 score margin 限制，避免每一轮都大幅重排 waiting queue。

4.5 Fairness 与 latency guard

引入 promotion 后，新的问题很快出现：高复用请求得到优先级后，decode-heavy、低复用和后台长输出请求的等待时间可能增加。这个问题在前后台混部实验中尤其明显。我们加入 age guard、head guard、defer max count、low-reuse age cap 和 latency-protected classes，使调度策略变成有界 foreground-priority。

guard 的目标是限制伤害范围。前台类可以在 queue pressure 下被提升，低复用请求可以短暂 defer，但超过年龄上限后必须被处理。decode-heavy/background 类在 foreground-priority stage 中可能回退，summary 必须单独报告它们的 p95 和 e2e p95。这个处理让实验结论更加诚实，也方便解释策略适用边界。

4.6 Run state、dashboard 与 replay

长测过程中，目录存在不代表实验完成。远程机器可能只跑完 LRU，KVFabric policy 还在运行；也可能 loadgen 停了，server 仍在；还可能 JSONL 很大，人工查看根本无法判断生命周期行为。为此加入 run_state.json、policy_state.json、heartbeat.json 和 rolling metrics。

dashboard 读取 run root，展示当前 stage、policy、运行时长、吞吐、延迟、prefix hit、evicted、rebuilt、admission、scheduler events 和 class metrics。replay 工具把 lifecycle JSONL 转成 block/family 变化过程，用于展示 LRU 与 KVFabric 在 cache pressure 下的差异。它们属于调试和展示工具，显著降低了长测定位成本。

4.7 Unified controller

中期策略名逐渐变多：shared_aware、family_protect、kvfabric_latency、kvfabric_admission、kvfabric_throughput。单靠名字无法说明 Admission、Eviction、Schedule 和 SLO protect 到底开了多强，也不利于不同 stage 保持同一套代码。后期把策略整理成 KVFabricControlConfig，所有 profile 都映射为连续参数。

最终 12h matrix 中，所有 stage 使用同一套 KVFabric 代码和统一 controller。差异来自 workload 和 stage-local 参数强调：prefill stage 强调 admission 与部分 eviction，interactive stage 强调 scheduler 和 SLO protect，enterprise 与 low-reuse stage 强调 admission guard。stage-local 环境变量放在 subshell 中，避免吞吐 profile 泄漏到 latency stage。

4.8 Admission 路径重构

远程高压实验暴露出一个关键问题：冷 RAG 或 burst 请求先写入 prefix cache，再被 eviction 驱逐，已经制造了 cache churn。更合理的做法是在写入阶段就降低低复用请求的 cache footprint。同时，早期多处限制缓存会导致 double-limit 风险，影响 block hash 和 cached blocks 的一致性。

后期重构把 admission limit 固定在 SingleTypeKVCacheManager.cache_blocks()。所有 full block 进入 prefix cache 前先通过唯一入口判断。durable/session/high priority 请求获得完整缓存机会，cold/low-reuse/bypass/unknown 请求根据 pressure 和 fraction 限制写入。这个改动是 final 12h 中 enterprise 和 low-reuse 结果的重要基础。

4.9 Latency header plumbing 与 scoring

latency 迭代初期，scheduler promotion 计数很低，SLO 和 turn 信息也没有正确进入 tracker。定位后发现 tracing header whitelist 漏掉了 SLO、session、turn-index 和 confidence 字段，trace loadgen 也没有在部分模式下显式发送 session/turn。修复后，lifecycle 事件中可以看到非零 hint_slo_ms 和 turn_index。

随后又发现 promotion 过粗：如果所有请求都有同一个 SLO，单纯按 SLO age promotion 会把后台和 decode-heavy 一起提升。最终改为带 class/hint 惩罚的评分选择，增加 latency scan window 和 latency max per step，给 short-output foreground class 加分，对 cold、decode、low-reuse、bypass 加 penalty。最终结果按前台交互类的 e2e p95 改善陈述，不把结论扩展成整体 30% latency 提升。

5. 实验设计与请求生成

5.1 实验平台

最终主要实验在远程 2 x RTX 3090 24GB 服务器上运行，模型为 Qwen/Qwen3.5-9B，tensor parallel size 为 2，max_model_len 为 4096，medium capacity profile 使用 GPU memory utilization 0.70、max_num_seqs 64、max_num_batched_tokens 24576，并开启 prefix caching。早期 27B 探索使用 Qwen/Qwen3.5-27B-FP8，在同一服务器上验证更紧张容量下的生命周期指标。

双卡部署由 vLLM tensor parallel 完成，模型权重按张量并行切分到两张 3090 上。KV Cache 随每个 tensor parallel rank 在本卡保存对应分片，两个 rank 共同服务同一批请求。KVFabric 的生命周期控制面运行在 vLLM Python 调度侧，记录的是逻辑 block 和控制面事件，不直接改写每张卡上的底层 KV tensor 布局。

最终 9B 实验环境

项目	设置
模型	Qwen/Qwen3.5-9B
GPU	2 x RTX 3090 24GB
并行方式	vLLM tensor parallel size = 2
vLLM	0.22.1 + KVFabric overlay
上下文长度	max_model_len = 4096
medium profile	gpu_memory_utilization = 0.70, max_num_seqs = 64, max_num_batched_tokens = 24576
接口	OpenAI-compatible /v1/chat/completions
结果根目录	experiments/long_pressure_benchmark/final_12h_results/2026-06-30_qwen3_5_9b_final_matrix

5.2 请求如何发送

所有正式请求都是网络形式发送。load generator 使用 httpx.AsyncClient 向 vLLM 服务端 POST `/v1/chat/completions`。payload 包含 model、messages、temperature 和 max_tokens；KVFabric 额外信息通过 HTTP headers 传入。这样可以保证实验走真实 OpenAI serving、scheduler、KVCacheManager 和 BlockPool 路径，避免离线直接调用内部函数造成路径偏差。

duration workload 使用闭环固定并发。客户端保持指定数量的 in-flight 请求，请求完成后从固定请求池中取下一条补上。这个模式适合吞吐和容量压力，因为更快的策略会在相同时间内完成更多请求。trace workload 使用开环 replay。每条请求有 scheduled_at_seconds，loadgen 到点发送，如果并发达到 max_in_flight 则通过 semaphore 排队。这个模式适合观察 e2e latency、send delay、SLO miss 和 foreground/background 竞争。

网络请求路径



5.3 请求类型设计

请求类型设计的核心是让有限代码产生足够多样的请求，同时保证复现性。我们不为每条请求手写 prompt；实现中定义 global_unit、tenant_unit、family_unit、sticky_unit、cold_unit、transient_unit、decode_unit 等文本片段，再按 tenant、family、session、turn 和随机种子组合。不同 class 使用不同重复次数、历史增长、max_tokens 和 hint。这样既能控制共享前缀位置，也能生成大量不会完全重复的冷内容。

Duration workload 请求类型

类型	构造方式	模拟场景	典型 max_tokens
durable_hot_family	global + tenant + family 共享模板，多轮重复回访	企业固定 workflow、共享业务规则、长期复用模板	64
sticky_session_followup	session 历史保留并按 turn 追加，family/session hint 稳定	多轮对话、项目上下文、连续 agent 工作流	48
cold_rag_unique	每条请求带唯一 evidence 段，expected reuse=none 或 low	一次性 RAG 查询、临时文档、工单片段	64
cold_rag_burst	相近时间发送多条冷请求，burst=true	突发检索流量、短时热点但后续不复用	64
transient_template_family	短期模板有部分相似，生命周期短	临时活动模板、短期报表格式	64
decode_heavy	prompt 较短或中等，输出长度长	内容生成、报告扩写、后台长输出	320

Trace workload 请求类型

类型	特征	用途
short_chat_qa	短输入短输出，前台交互类	观察轻量交互请求在队列压力下的 e2e 延迟
tenant_workflow_hot	租户固定流程和高复用 family	验证 durable 工作流的 cache affinity
agent_tool_loop	工具协议、观察结果和后续动作混合	模拟 agent 多步任务中的复用和调度
project_code_followup	项目上下文和代码片段多轮回访	验证长前缀复用与前台 p95
deep_multi_turn_chat	历史逐轮增长，session/turn 明确	验证多轮长对话的 prefix reuse
long_doc_research_followup	长文档研究和后续追问	验证较长前缀的缓存价值
background_cold_lookup	后台冷检索，reuse 低	作为 foreground-priority 的 guard class
decode_heavy_background	长输出后台任务	观察 scheduler promotion 对 decode-heavy 的影响
rag_qa_hot_docs / cold_docs	RAG 文档热/冷 family	企业混合流量中的热文档与冷文档对照
extraction_classification	短输出结构化抽取	测试低成本请求在混合服务中的排队影响
single_turn_api_task	单轮 API 风格任务	低复用、低频 guard

不重复性来自四个层次。第一，tenant、client、family、session id 由 seed 控制但组合多样；第二，prompt 由不同 unit、重复次数、字段名、时间戳、证据片段和 turn history 拼接；第三，trace 的 scheduled_at_seconds 带 burst、wave 和 session interval；第四，max_tokens 按 class 分布采样。对于 A/B，seed 和 trace hash 固定，保证两种策略看到同一请求计划。

5.4 指标口径

主要指标定义

指标	含义	使用场景
Req/s	成功完成请求数 / 计分秒数	吞吐基础指标
Total tok/s	成功输出和输入相关 token 吞吐汇总	整体服务压力
Goodput tok/s	SLO 内成功完成的 token 数 / 计分秒数	SLO 边界下的有效吞吐

指标	含义	使用场景
Service latency P50/P95/P99	服务端请求处理时间分位数	duration 和 trace 都可使用
E2E latency P50/P95/P99	open-loop 中包含 send delay/排队的端到端时间	scheduler、拥塞和 SLO 分析
Prefix hit / eligible hit	prefix cache 命中和可命中 token 比例	复用质量
Warm-family hit	具有 family 复用证据的命中比例	共享主干质量
Evicted / rebuilt	驱逐 block 数和被驱逐后同 hash 重建数	错误驱逐和策略反馈
Admission limited / saved blocks	被限制缓存的请求或 block 数	admission 是否减少 churn
Scheduler promotions	positive 或 latency promotion 事件数	调度策略是否实际起效

5.5 最终 12h 实验矩阵

最终矩阵由四个正式 stage 组成。每个 stage 都是 LRU 与 KVFabric 的 A/B 对比，使用同一套代码和统一 controller，差异来自 workload 和 stage-local preset。矩阵总 loadgen 时间约 660 分钟，加上服务启动、关闭、同步和 summary，接近 12 小时 wall-clock。

最终 12h Matrix

Stage	类型	配置	主要场景	主指标
prefill_throughput_medium	duration	qwen3_5_9b_prefill_throughput_medium.json	稳定共享前缀 + cold RAG/burst 冲击 + durable revisit	selected SLO goodput, rebuilt
interactive_latency_medium	trace	qwen3_5_9b_foreground_latency_background_90m.json	前台交互请求与后台 cold/decode 混部	foreground e2e P95, latency promotions
enterprise_normal_medium	trace	qwen3_5_9b_enterprise_normal_75m.json	多租户企业混合服务，hot/cold RAG、agent、support、decode	goodput, e2e P95, admission saved
low_reuse	trace	qwen3_5_9b_low_reuse_45m.json	低频、低复用、无稳定共享前缀请求	non-regression, evicted, admission saved

6. 测试结果与分析

6.1 总体结果

最终 12h 矩阵的主结果如下。prefill throughput 是最清晰的高压吞吐证据，KVFabric 在 40s selected SLO probe 下 goodput 从 1815.04 tok/s 提升到 3590.21 tok/s，rebuilt-from-eviction 从 11790 降到 1725。interactive latency 需要按 foreground-priority 解读，整体 e2e P95 从 273.888s 降到 221.542s，六个前台类 e2e P95 均超过 30% 改善。enterprise normal 和 low-reuse 的主证据来自 admission 降低低价值 cache 写入和排队压力。

最终 12h 矩阵主结果

Stage	KVFabric profile	主指标	LRU	KVFabric	变化
Prefill	kvfabric_throughput	selected SLO goodput tok/s	1815.04	3590.21	+97.80%
Prefill	kvfabric_throughput	rebuilt-from-eviction	11790	1725	-85.37%
Latency	kvfabric_latency	overall e2e P95 latency	273.888s	221.542s	-19.11%
Latency	kvfabric_latency	e2e goodput tok/s	106.38	305.23	+186.94%
Enterprise	kvfabric_admission	goodput tok/s	1383.78	1624.16	+17.37%
Enterprise	kvfabric_admission	e2e P95 latency	316.015s	112.722s	-64.33%
Low reuse	kvfabric_admission	goodput tok/s	317.95	448.88	+41.18%
Low reuse	kvfabric_admission	e2e P95 latency	325.620s	15.798s	-95.15%

生命周期证据汇总

Stage	Prefix hit	Warm-family hit	Evicted blocks	Rebuilt-from-eviction
Prefill	21.28% -> 30.93%	41.41% -> 71.11%	81330 -> 54030	11790 -> 1725 (-85.37%)
Latency	8.56% -> 10.54%	15.31% -> 18.85%	30915 -> 29902	10845 -> 9788 (-9.75%)
Enterprise	3.98% -> 5.03%	10.30% -> 13.01%	30377 -> 22321	6602 -> 6455 (-2.23%)
Low reuse	0.00% -> 0.00%	0.00% -> 0.00%	4348 -> 777	0 -> 0

6.2 Stage 1: Prefill Throughput

Stage 1 使用 duration workload，模拟稳定共享工作集在冷流量冲击下的缓存竞争。实验分三段：working_set_warmup 用 900s 和并发 32 建立 durable hot family 与 sticky session；cold_churn_main 用 4500s 和并发 64 注入大量 cold RAG、burst、transient 和少量 decode-heavy；durable_revisit 用 1800s 和并发 68 回访热 family 和 sticky session，检查之前建立的共享前缀是否还留在 cache 中。

Stage 1 实验设置

Segment	时长	并发	作用
working_set_warmup	900s	32	建立 durable hot family 和 sticky session 工作集，不作为主要计分段
cold_churn_main	4500s	64	大量 cold RAG / burst / transient 进入，制造 cache churn 和 eviction pressure
durable_revisit	1800s	68	热 family 与 sticky session 回访，验证共享前缀是否被保留

该 stage 的核心逻辑是：LRU 会把低复用长 prompt 写入大量 full blocks，cold RAG 和 burst 在短时间内占用 prefix cache，后续 durable revisit 需要重新 prefill；KVFabric 通过 admission 限制 hint_bypass 和 hint_low_reuse 请求的缓存写入，同时保留 durable/high reuse 的完整缓存机会，减少

热共享主干被短期冷流量冲掉的概率。

Stage 1 主要结果

指标	LRU	KVFabric	变化
Requests	11520	12105	+5.08%
Req/s	1.5622	1.6391	+4.92%
Selected SLO goodput tok/s (40s)	1815.04	3590.21	+97.80%
P50 latency	40.925s	32.649s	-20.22%
P95 latency	45.707s	35.977s	-21.29%
Prefix hit	21.28%	30.93%	+45.35% relative
Warm-family hit	41.41%	71.11%	+71.72% relative
Evicted blocks	81330	54030	-33.57%
Rebuilt-from-eviction	11790	1725	-85.37%
Admission limited	0	26115	机制生效
Saved blocks	0	68475	减少低价值缓存写入

结果表明，Stage 1 的收益与生命周期证据能够对齐。prefix hit 和 warm-family hit 同时上升，evicted 和 rebuilt 大幅下降，说明减少 cold churn 之后，热 family 的回访更容易命中已有 prefix。selected SLO goodput 使用 40s probe，raw metrics 仍保留 18s、20s、22s、24s、28s、32s 和 40s 全部阈值。报告展示 40s 是因为该阈值对应本阶段的 SLO 边界和最大相对提升。

6.3 Stage 2: Interactive Latency

Stage 2 使用 open-loop trace，profile 为 daily_dedicated_reuse，duration 5400s，target request rate 0.9，max_in_flight 96，SLO 60s。trace 中 68.76% 请求属于 session，包含 short_chat_qa、tenant_workflow_hot、agent_tool_loop、project_code_followup、deep_multi_turn_chat、long_doc_research_followup 六个前台类，并独立注入 background_cold_lookup 和 decode_heavy_background 作为 guard。

Stage 2 实验设置

项目	内容
Trace profile	daily_dedicated_reuse
时长与速率	5400s, target 0.9 req/s, actual 0.927 req/s
并发控制	max_in_flight 96, vLLM max_num_seqs 32 的队列压力配置
前台类	agent_tool_loop、deep_multi_turn_chat、long_doc_research_followup、project_code_followup、short_chat_qa、tenant_workflow_hot
后台类	background_cold_lookup、decode_heavy_background
主要机制	latency promotion 3442 次，按 class/hint/age/short-output 评分

Stage 2 总体结果

指标	LRU	KVFabric	变化
Requests / errors	4162 / 11	4162 / 11	相同
Req/s	0.5434	0.5836	+7.40%
Goodput tok/s	106.38	423.34	+297.96%
E2E goodput tok/s	106.38	305.23	+186.94%
Service P50	132.337s	69.757s	-47.29%

指标	LRU	KVFabric	变化
Service P95	223.615s	203.591s	-8.95%
E2E P50	153.822s	80.727s	-47.52%
E2E P95	273.888s	221.542s	-19.11%
Warm-family hit	15.31%	18.85%	+23.12% relative
Rebuilt-from-eviction	10845	9788	-9.75%
Latency promotions	0	3442	机制生效

Stage 2 前台类 e2e P95

前台请求类	LRU e2e P95	KVFabric e2e P95	变化
agent_tool_loop	246.209s	136.808s	-44.43%
deep_multi_turn_chat	242.968s	133.376s	-45.11%
long_doc_research_followup	245.434s	150.830s	-38.55%
project_code_followup	266.814s	178.573s	-33.07%
short_chat_qa	196.477s	91.316s	-53.52%
tenant_workflow_hot	218.928s	89.736s	-59.01%

Stage 2 的结论需要按 class 解释。它证明了 foreground-priority 在队列压力下能保护前台交互类，六个前台类 e2e P95 均下降 30% 以上；同时 background_cold_lookup 和 decode_heavy_background 的 e2e P95 有回退。这是策略选择的代价，需要在报告中明确说明。最终报告把该 stage 定义为前台优先调度实验，结论范围限于前台类保护。

6.4 Stage 3: Enterprise Normal

Stage 3 使用 enterprise_mixed trace, duration 4500s, target request rate 0.75, actual request rate 0.922, max_in_flight 48, SLO 90s。trace 包含 10 个 tenants、64 个 clients、917 个 families, session request ratio 56.81%, burst ratio 3.49%。请求类型包括 agent_tool_loop、multi_turn_support、rag_qa_hot_docs、rag_qa_cold_docs、tenant_workflow_hot、decode_heavy_report 和 extraction_classification。

Stage 3 实验设置

项目	内容
Trace profile	enterprise_mixed
时长与速率	4500s, target 0.75 req/s, actual 0.922 req/s
规模	10 tenants, 64 clients, 917 families
复用结构	session request ratio 56.81%, partial hints
主要混合	agent tool loop、multi-turn support、hot/cold RAG docs、decode-heavy report、extraction/classification
主要机制	admission dominant, 限制 low-reuse、bypass、unknown 类 cache 写入

Stage 3 主要结果

指标	LRU	KVFabric	变化
Requests / errors	4071 / 0	4071 / 0	相同
Req/s	0.8635	0.9098	+5.36%
Goodput tok/s	1383.78	1624.16	+17.37%

指标	LRU	KVFabric	变化
E2E goodput tok/s	520.11	1600.36	+207.69%
Service P50	39.391s	31.065s	-21.14%
Service P95	139.672s	107.189s	-23.26%
E2E P50	136.024s	33.791s	-75.16%
E2E P95	316.015s	112.722s	-64.33%
Evicted blocks	30377	22321	-26.52%
Rebuilt-from-eviction	6602	6455	-2.23%
Admission limited / saved	0 / 0	1463 / 5355	机制生效

Stage 3 的 prefix hit 提升不大，rebuilt 下降也只有 2.23%，所以主要收益不能简单归因于共享前缀复用。更合理的解释是 admission 控制了 rag_qa_cold_docs、decode_heavy_report 和 extraction_classification 等低价值缓存写入，降低 evicted blocks 和 queue backlog，使 open-loop 下的 e2e latency 明显改善。这个 stage 证明 KVFabric 在普通企业混合流量下具备泛化价值和低错误率。

6.5 Stage 4: Low Reuse Guard

Stage 4 使用 low_reuse_low_frequency trace, duration 2700s, target request rate 0.25, actual request rate 0.2748, max_in_flight 16, SLO 120s。该 trace 的 session request ratio 为 0, cold family reuse probability 接近 0, unique families 为 648。请求类型主要是 decode_heavy_background、extraction_classification、rag_qa_cold_docs、short_chat_qa 和 single_turn_api_task。

Stage 4 实验设置

项目	内容
Trace profile	low_reuse_low_frequency
时长与速率	2700s, target 0.25 req/s, actual 0.2748 req/s
并发控制	max_in_flight 16, SLO 120s
复用结构	session request ratio 0.00%, prefix hit 预期为 0
主要请求	decode_heavy_background、extraction_classification、rag_qa_cold_docs、short_chat_qa、single_turn_api_task
主要机制	admission guard, 限制 low/unknown/bypass 写入 prefix cache

Stage 4 主要结果

指标	LRU	KVFabric	变化
Requests / errors	727 / 0	727 / 0	相同
Req/s	0.2496	0.2751	+10.22%
Goodput tok/s	317.95	448.88	+41.18%
E2E goodput tok/s	153.28	448.88	+192.86%
Service P50	29.178s	3.361s	-88.48%
Service P95	174.143s	15.796s	-90.93%
E2E P50	151.216s	3.362s	-97.78%
E2E P95	325.620s	15.798s	-95.15%
Prefix hit / warm-family	0.00% / 0.00%	0.00% / 0.00%	无复用
Evicted blocks	4348	777	-82.13%

指标	LRU	KVFabric	变化
Admission limited / saved	0 / 0	863 / 2293	减少低价值 cache footprint

Stage 4 是边界验证。这里没有 prefix hit，也没有 rebuilt-from-eviction，所以它不证明 prefix reuse。它证明的是：当请求基本没有复用价值时，hint-aware admission 可以减少无意义 cache 写入和驱逐，把系统从大量低价值 cache churn 中解放出来。该 run 中 latency 和 goodput 提升很大，但报告中仍按 guardrail 场景表述，避免把低复用结果包装成共享前缀收益。

6.6 早期 27B 探索结果

27B 结果主要用于早期远程探索和压力特征判断。Qwen3.5-27B-FP8 在 2 x RTX 3090 上显存更紧张，适合观察 LRU 在容量压力下的 rebuilt 行为。27B saturation 12h 中，shared-aware goodput 从 601.57 提升到 675.80 tok/s，rebuilt 从 12680 降到 3566；4h saturation 中 shared-aware 提升 16.96%，family-protect 提升 5.07%。enterprise trace 中吞吐接近，但 rebuilt 分别下降 52.71% 和 64.60%。sticky conversation trace 中 family-protect 多次保持约 80% rebuilt 降幅，但也暴露了 cold/decode noise 的尾延迟风险。

这些 27B 实验帮助我们确认两个方向：第一，rebuilt-from-eviction 是解释错误驱逐的强指标；第二，强保护和强驱逐排序必须有 guard，否则容易在后台和冷请求上付出尾延迟代价。最终主验收改用 Qwen3.5-9B，是因为 9B 能在 2 x 3090 上跑更稳定的中等容量矩阵，并留出足够空间观察不同策略的差异。

6.7 测试结论

综合四个 stage，KVFabric 的收益具有明确场景边界。高压共享前缀和冷流量混合时，admission 与共享保护能显著减少 rebuilt，提升 SLO goodput。前后台混部时，scheduler scoring 能保护前台交互请求，但需要接受后台/decode 的可解释代价。普通企业混合流量中，admission 的价值更多体现为减少 cache churn 和 backlog。低复用场景中，prefix reuse 指标为零，KVFabric 的目标是控制低价值缓存写入和保证非回归。

7. 调试工具与可视化

7.1 工具链总览

远程长测让调试问题发生了变化：单次运行可能持续数小时，事件日志可能非常大，实验结果涉及服务端、loadgen、Prometheus、lifecycle、trace 和 summary 多条路径。KVFabric 为此建立了调试工具组合，先确认 run 是否完成，再解释结果来自哪条机制。

调试工具链

工具	输入	输出/用途
run_state / policy_state	stage、policy、退出码、时间戳	区分 completed、failed、partial、stalled
heartbeat	运行中状态、当前 policy、最近更新时间	判断远程长测是否卡住
Lifecycle JSONL	block/request/family 事件流	定位 prefix lookup、sealed、touched、evicted、rebuilt、promotion
Prometheus summary	服务端指标采样	观察 cache pressure、throughput、latency、metadata overhead
online metrics	loadgen 侧 metrics.json、class_metrics、segment_metrics	计算 req/s、goodput、SLO、service/e2e latency
summary generator	Prometheus + JSONL + online metrics	生成统一 Markdown/JSON 表格
dashboard	run root 只读读取	实时查看 rolling metrics、class metrics、policy 状态
replay / GIF	lifecycle JSONL	可视化 KV block 生命周期和 eviction 差异

7.2 Lifecycle JSONL 的调试价值

Lifecycle JSONL 是项目中最重要调试证据。它把一次请求从 request_hints_observed、prefix_lookup、block_allocated、block_sealed、block_touched、ref_count_changed 到 block_evicted 的过程记录成事件。header plumbing 调试时，我们通过 JSONL 确认 hint_slo_ms、session id 和 turn_index 是否进入 tracker；admission 调试时，通过 cache_admission_limited 事件确认受限 class 和 saved blocks；scheduler 调试时，通过 request_latency_promoted 和 request_promoted 事件确认 promotion 是否真正发生。

Lifecycle JSONL 典型事件

类别	事件	能定位的问题
请求语义	request_hints_observed	class、session、SLO、priority 是否进入 vLLM
prefix cache	prefix_lookup	hit tokens、first miss depth、eligible hit 口径
block 生命周期	block_allocated / block_sealed / block_touched / block_evicted	active -> sealed -> shared -> evicted 路径
admission	cache_admission_limited	低价值请求被限制了多少 block，原因是什么
scheduler	request_promoted / request_latency_promoted / request_deferred	waiting queue 策略是否实际起效
重建反馈	rebuilt_from_eviction	错误驱逐后同 hash 是否重新 sealed

7.3 Dashboard 和 replay

dashboard 是长测期间的在线观察入口。它读取 run root 中的 rolling metrics、Prometheus summary、lifecycle metrics、trace summary 和 job log，不修改实验结果。页面重点展示当前 stage、policy、吞吐、延迟、KV pressure、prefix hit、evicted、rebuilt、admission limited、scheduler

promotion 和 class-level latency。出现 partial 或 stalled 时，dashboard 可以直接指出停在哪个 policy、最后 heartbeat 时间和最近错误。

replay 工具把 lifecycle JSONL 转为可播放的 KV cache 状态变化，适合汇报展示和事后复盘。与静态 summary 相比，replay 能直观看看到 hot family 主干在 LRU 中被冷流量挤出，以及 KVFabric admission/family-protect 如何减少低价值 block 进入或延后高价值 block 被驱逐。

8. 项目总结与展望

KVFabric 最终完成了一个围绕大模型推理服务 KV Cache 生命周期管理的系统原型。项目从 vLLM baseline 阅读和本地 smoke 开始，逐步加入 lifecycle side table、JSONL 事件、Prefix Family、evicted shadow、shared-aware eviction、family-protect、hint-aware admission、scheduler affinity、latency guard、Prometheus metrics、summary、dashboard 和 replay。后期又把实验系统扩展到远程 2 x RTX 3090 上的 Qwen3.5-9B 长周期矩阵，形成可复现的最终测试结果。

从工程结果看，KVFabric 的价值主要体现在三个方面。第一，它让 KV Cache 管理过程可观测，能从 block、request、family、class 和 SLO 多个层次解释结果。第二，它提供了可调的控制面策略，在共享前缀和容量竞争场景中减少错误驱逐和重算，在低复用场景中减少无意义 cache churn。第三，它把操作系统课程中的页生命周期、工作集、准入控制、置换策略和调度反馈迁移到模型原生推理服务中，形成可运行、可测试、可展示的系统。

项目仍有清晰边界。当前实现没有修改底层 KV tensor 数据面，没有实现 token 级 trie 或任意 chunk 级共享，也没有做真实 CoW。Prefix Family 是基于 vLLM full-block hash 链的控制面元数据。策略收益随 workload 变化，报告中按场景给出结论：prefill throughput 是强复用和 SLO goodput 证据；interactive latency 是 foreground-priority 证据；enterprise normal 主要体现 admission 和 backlog 改善；low-reuse 是 guardrail 与 cache churn 控制。

后续可以继续推进的方向

- 把 Prefix Family 从 block 级近似扩展到更细粒度的结构统计，记录 child branch 权重分布和更完整的 family entropy。
- 在更大模型和更多 GPU 拓扑上评估 controller 参数是否稳定，尤其是 TP/PP、显存利用率和 max_num_seqs 对策略的影响。
- 将 request hint 与真实上层网关或 agent runtime 对接，替代实验 loadgen 中的人工 header。
- 把 dashboard/replay 与 Prometheus/Grafana 或更轻量的静态报告结合，形成长期回归测试入口。
- 为 admission、eviction 和 scheduler 建立自动参数搜索或 workload-aware preset，减少人工调参成本。
- 补充更严格的 fixed-work A/B 子实验，进一步审计闭环 fixed-concurrency 的 completed mix drift。

参考资料

本报告引用的外部材料主要用于说明 LLM serving、PagedAttention、prefix caching、工作负载建模和 SLO goodput 的研究背景。项目事实、代码路径、实验结果和数值均以本仓库归档内容为准，尤其是 `docs/current`、`experiments/long\_pressure\_benchmark`、最终 12h run root 和四个 summary 文件。

- Kwon et al., Efficient Memory Management for Large Language Model Serving with PagedAttention.
- vLLM 0.22.1 source code and prefix caching design documentation.
- BurstGPT: A Real-world Workload Dataset to Optimize LLM Serving Systems.
- ServeGen: Workload Modeling for Large Language Model Serving.
- Sarathi-Serve and DistServe papers on prefill/decode scheduling and SLO-oriented serving.
- SGLang / RadixAttention documentation and blog materials on structured generation and prefix reuse.
- KVFabric 仓库 README、docs/current 设计文档、experiments/long_pressure_benchmark final 12h results、dashboard/replay design notes。

仓库内主要事实来源

材料	用途
README.md	项目定位、当前状态、成员分工、主要机制、实验体系和最终 PPT 链接
docs/current/kvfabric_final_code_design_vs_vllm_2026-06-30.md	当前代码设计、与 vLLM 的差异、核心文件和策略边界
docs/current/kvfabric_9b_final_matrix_and_latency_iteration_2026-06-30.md	9B 吞吐与 latency 迭代过程、controller 参数和 final matrix 收敛过程
docs/current/kvfabric_12h_acceptance_experiment_design.md	长测验收方案、压力校准、SLO goodput、27B acceptance 设计
experiments/long_pressure_benchmark/README.md	长测工具链、loadgen、trace 字段、远程脚本和 artifact 约定
final_12h_results/*/summaries/*.md	最终四个 stage 的主表、class metrics、lifecycle evidence 和 notes
final_12h_results/*/analysis/final_12h_sync_and_analysis.md	最终结果同步状态、主结论、生命周期证据和谨慎表述
docs/endterm/KVFabric期末汇报.pptx	期末汇报结构、迭代主线、工具链介绍、实验设计和结果展示

附录 A. 关键实现细节补充

KVFabric 的开关集中在环境变量中。`KVFABRIC\_ENABLE` 控制总开关，`KVFABRIC\_PROFILE` 选择 profile，`KVFABRIC\_ADMISSION\_STRENGTH`、`KVFABRIC\_EVICTION\_STRENGTH`、`KVFABRIC\_SCHEDULER\_STRENGTH`、`KVFABRIC\_SLO\_PROTECTION\_STRENGTH` 控制四类机制，`KVFABRIC\_HINT\_TRUST` 控制 hint 权重，`KVFABRIC\_EVICTION\_SELECTOR` 控制 rank 或 linear 选择器，`KVFABRIC\_LRU\_ADMISSION\_POLICY` 控制 LRU 对照下是否保留原始全缓存语义。

报告中的 LRU 对照并不完全关闭观测路径。为了拿到 lifecycle 指标，LRU policy 也可以启用 tracker，但 controller strength 为 0，不改变 victim 选择、admission 和 scheduler 行为。这样能够在同一套 summary 中比较 LRU 与 KVFabric 的 prefix hit、evicted、rebuilt 和 class metrics。

KVFabric 的 request hints 使用 header 传递，是实验控制面和业务控制面之间的接口。它没有改变 OpenAI ChatCompletions 的 messages 语义，也没有要求模型看到这些字段。服务端把 header 解析成 KVFabricRequestHints 后，主要影响 admission 和 scheduler。hint 丢失时，系统会回退到 unknown/normal/default 路径。

核心事件接入点

接入点	触发事件	用途
BlockPool.__init__	tracker_initialized	建立 lifecycle tracker 和 JSONL writer
BlockPool.cache_full_blocks()	block_sealed	记录 full block hash、depth、parent/root 和 rebuilt 标记
BlockPool.touch()	block_touched	记录 prefix hit、share/ref count 和 family touch
BlockPool.free_blocks()	ref_count_changed	更新 active、shared、cooling 状态
BlockPool._maybe_evict_cached_block()	block_evicted	写入 EvictedShadow，记录 retain score 和 eviction time
KVCacheManager.get_computed_blocks()	prefix_lookup	记录 prompt tokens、hit tokens 和 first miss depth
SingleTypeKVCacheManager.cache_blocks()	cache_admission_limited	控制低价值请求写入 prefix cache 的深度
Scheduler.schedule()	request_promoted / request_latency_promoted / request_deferred	把 cache affinity 和前台延迟保护转成调度行为
serving.py + tracing/utils.py	request_hints_observed	把 request class、tenant、family、session、turn 和 SLO 传入控制面

主要环境变量组

变量组	代表变量	说明
总开关与 profile	KVFABRIC_ENABLE, KVFABRIC_PROFILE	控制 overlay 是否改变策略行为，以及选择 admission/throughput/latency 等 preset
Controller strength	KVFABRIC_ADMISSION_STRENGTH, KVFABRIC_EVICTION_STRENGTH, KVFABRIC_SCHEDULER_STRENGTH, KVFABRIC_SLO_PROTECTION_STRENGTH	四个连续控制维度，便于同一代码在不同 stage 中强调不同机制
Admission fraction	KVFABRIC_LOW_REUSE_CACHE_FRACTION, KVFABRIC_TRANSIENT_CACHE_FRACTION, KVFABRIC_BYPASS_CACHE_FRACTION, KVFABRIC_DURABLE_CACHE_FRACTION, KVFABRIC_COLD_CACHE_FRACTION	按 expected reuse 和 cache priority 控制可缓存 full block 比例

变量组	代表变量	说明
Eviction selector	KVFABRIC_EVICTION_SELECTOR, KVFABRIC_EVICTION_CANDIDATE_WINDOW, KVFABRIC_RECOMPUTE_SCORE_WEIGHT	控制候选窗口、linear/rank 选择器和重算代价权重
Scheduler scan	KVFABRIC_SCHEDULER_POSITIVE_SCAN_WINDOW, KVFABRIC_SCHEDULER_LATENCY_SCAN_WINDOW, KVFABRIC_SCHEDULER_POSITIVE_MAX_PER_STEP, KVFABRIC_SCHEDULER_LATENCY_MAX_PER_STEP	控制 waiting queue 扫描范围和每轮最多 promotion 数
Latency guard	KVFABRIC_SCHEDULER_LATENCY_PROTECTED_CLASSES, KVFABRIC_SCHEDULER_LATENCY_PROTECTED_PROMOTE_AGE_MS, KVFABRIC_SCHEDULER_HEAD_AGE_GUARD_MS	定义前台类、promotion 年龄和队首保护条件

附录 B. 最终实验配置摘要

最终配置摘要

配置	关键参数
prefill_throughput_medium	concurrency 64; max_tokens 48; segments 900s/4500s/1800s; SLO probes 18/20/22/24/28/32/40s; hot_family_count 8; sticky_session_count 24
interactive_latency_medium	duration 5400s; rate 0.9; max_in_flight 96; background_mix_probability 0.20; SLO 60s; foreground classes 6 类
enterprise_normal_medium	duration 4500s; rate 0.75; max_in_flight 48; tenant_count 10; client_count 64; hot_family_count 48; SLO 90s
low_reuse	duration 2700s; rate 0.25; max_in_flight 16; tenant_count 12; client_count 96; session_reuse_probability 0.10; SLO 120s

最终 stage 的请求与输出长度

Stage	请求规模	主要 max_tokens 设置	发送方式
prefill_throughput_medium	duration 请求池按 1200 rounds 生成, 正式计分完成 11520/12105 请求	generation max_tokens 48; durable/cold/transient 通常 64; sticky 48; decode-heavy 320	closed-loop fixed concurrency, 分段并发 32/64/68
interactive_latency_medium	trace requests 5006, 最终完成 4162, session ratio 68.76%	按 class 从长度分布采样, 前台短/中输出, decode_heavy_background 长输出	open-loop replay, scheduled_at_seconds 到点发送, max_in_flight 96
enterprise_normal_medium	trace requests 4149, 最终完成 4071, families 917	RAG/agent/support 中等输出, decode_heavy_report 长输出, extraction 短输出	open-loop replay, max_in_flight 48
low_reuse	trace requests 742, 最终完成 727, session ratio 0.00%	低频单轮任务、冷 RAG、decode-heavy 后台任务按 class 分布采样	open-loop replay, max_in_flight 16

最终 stage 的 hint 分布要点

Stage	Hint 覆盖	主要 hint 结构
prefill_throughput_medium	100.00%	high/durable 覆盖 durable_hot_family 与 sticky_session; low/none 覆盖 cold_rag_unique; bypass 覆盖 cold_rag_burst; transient 覆盖 transient_template_family
interactive_latency_medium	100.00%	partial_hints; 前台 session/family/turn/SLO 可见; 后台类带 low/bypass/none 等低复用语义
enterprise_normal_medium	100.00%	10 tenants、64 clients、770 hint families; durable 3261, none 635, unknown 253
low_reuse	100.00%	无 hint families; low 193, normal 548, bypass 1; expected reuse 以 none/unknown 为主

附录 C. 报告结论边界

本报告采用按场景陈述的方式。prefill throughput 阶段可以陈述 selected SLO goodput +97.80%、rebuilt -85.37%，因为吞吐、hit、warm-family、evicted 和 rebuilt 证据链一致。interactive latency 阶段可以陈述六个前台类 e2e P95 均超过 30% 改善，并同步说明后台类回退。enterprise normal 阶段可以陈述 goodput 和 e2e P95 改善，原因偏向 admission 降低 churn 和 backlog。low-reuse 阶段可以陈述无 prefix reuse 下的 admission guard 效果，不能把它表述成 prefix reuse 胜利。

这个边界是项目可信度的一部分。KV Cache 生命周期管理更像操作系统中的缓存和调度策略，需要根据工作集、容量压力、SLO 和请求类别解释。KVFabric 的贡献在于让这些策略可记录、可调节、可复盘，并在真实 vLLM 服务路径上展示可观收益。

结果陈述边界

结论	可引用证据	需要同步说明的限制
高压共享前缀下 SLO goodput 显著提升	prefill selected SLO 1815.04 -> 3590.21 tok/s; rebuilt 11790 -> 1725; warm-family hit 41.41% -> 71.11%	selected probe 为 40s; raw JSON 保留全部 SLO probe
前台交互类 e2e P95 明显改善	六个前台类 e2e P95 改善 33.07% 到 59.01%; latency promotions 3442	background_cold_lookup 与 decode_heavy_background 有回退，属于 foreground-priority profile 的代价
企业混合流量中 admission 有泛化收益	goodput +17.37%; e2e P95 -64.33%; evicted 30377 -> 22321; admission saved 5355 blocks	rebuilt 只下降 2.23%，收益主要来自 churn/backlog 控制
低复用场景通过 admission guard 控制 cache footprint	prefix hit 为 0; evicted 4348 -> 777; admission saved 2293 blocks; e2e P95 325.620s -> 15.798s	该 stage 不作为 prefix reuse 证明

后续评估风险与处理方式

风险	处理方式
闭环 duration 中更快策略完成更多请求，可能造成 completed mix drift	固定 seed 和全局 shuffle 请求池; summary 审计 class/session/family 占比; 必要时补 fixed-work 子实验
open-loop trace 在压力不足时 raw tok/s 接近	关注 e2e P95、SLO miss、send delay、queue backlog 和 class-level latency
强 promotion 可能伤害后台/长输出请求	增加 latency-protected classes、head age guard、defer cap，并单独报告 guard classes
强 eviction 排序可能增加热路径开销	保留无压力快路径，9B profile 使用 linear selector 和候选窗口限制
hint 缺失或噪声可能影响策略判断	支持 unknown/default fallback; 实验保留 no_hints、partial_hints、full_hints 和 noisy_hints 设计口径
JSONL 过大影响同步和归档	默认同步 lifecycle metrics 和采样结果，完整 JSONL 仅在 replay 或深度调试时保留